

# **LAPORAN PRAKTIKUM**

**Aplikasi Berbasis Platform**

**MODUL 12 & 13**

Laravel Database & Git Branch



**Disusun oleh:**

Muhammad Rafiful Hana

2311102227

S1 IF-11-REG04

**Dosen Pengampu:**

Cahyo Prihantoro, S.Kom., M.Eng

**PROGRAM STUDI S1 INFORMATIKA  
FAKULTAS INFORMATIKA  
TELKOM UNIVERSITY PURWOKERTO  
2026**

## DASAR TEORI

Modul 12 membahas penggunaan database pada *Laravel* melalui pembuatan aplikasi *CRUD* (*Create, Read, Update, Delete*) produk. *CRUD* merupakan konsep dasar dalam pengelolaan data pada sistem basis data yang memungkinkan pengguna untuk menambah, menampilkan, mengubah, dan menghapus data. Pada *Laravel*, proses ini didukung oleh fitur *migration* yang berfungsi untuk membuat dan mengelola struktur tabel database secara terstruktur melalui kode, sehingga perubahan skema database dapat dikontrol dengan baik.

Selain *migration*, *Laravel* juga menggunakan *model* sebagai representasi tabel database dalam bentuk objek yang memudahkan interaksi dengan data menggunakan *Eloquent ORM*. Model ini bekerja sama dengan *controller* untuk mengatur logika aplikasi, serta *view* untuk menampilkan data kepada pengguna. Proses komunikasi antara komponen tersebut diatur melalui *route resource* yang menyederhanakan pembuatan endpoint *CRUD*. Dalam pengembangannya, *Laravel* juga menyediakan fitur *templating Blade* untuk mengelola tampilan serta *validasi form* untuk memastikan data yang dimasukkan pengguna sesuai dengan aturan yang ditentukan.

Modul 13 membahas fitur lanjutan *Laravel* yang mencakup *session*, *middleware*, *autentikasi*, dan *relasi model*. *Session* digunakan untuk menyimpan data sementara pengguna selama interaksi dengan aplikasi, sedangkan *middleware* berfungsi sebagai lapisan penyaring permintaan (request) sebelum mencapai controller, misalnya untuk autentikasi pengguna. Fitur *autentikasi* digunakan untuk mengelola proses login, registrasi, dan otorisasi pengguna dalam sistem. Sementara itu, *relasi model* memungkinkan pengelolaan hubungan antar tabel database seperti one-to-one, one-to-many, dan many-to-many, sehingga struktur data menjadi lebih terorganisir dan mudah dikelola dalam aplikasi berbasis *Laravel*.

## GUIDED

### MODUL 12

#### A. Konfigurasi Database

Konfigurasi database dilakukan pada file **.env** untuk menghubungkan Laravel dengan MySQL.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ecommerce
DB_USERNAME=root
DB_PASSWORD=
```

Pengaturan tersebut digunakan agar Laravel dapat terkoneksi dengan database bernama **ecommerce**. Setelah konfigurasi benar, Laravel dapat melakukan proses penyimpanan, pengambilan, pembaruan, dan penghapusan data pada database tersebut.

#### B. Migration Product

Migration digunakan untuk membuat struktur tabel database secara terstruktur menggunakan kode Laravel.

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->integer('price');
    $table->timestamps();
});
```

Pada tabel **products**, dibuat beberapa kolom utama yaitu **id**, **name**, **price**, serta timestamp otomatis. Migration memungkinkan pengelolaan struktur database menjadi lebih terkontrol dan mudah dikembangkan, terutama ketika terjadi perubahan skema data pada aplikasi.

#### C. Model Product

```
class Product extends Model
{
    protected $fillable = ['name', 'price'];
}
```

Model digunakan sebagai penghubung antara aplikasi Laravel dan tabel database. Dalam model Product, properti **fillable** digunakan untuk menentukan kolom yang diperbolehkan menerima input secara mass assignment. Hal ini bertujuan untuk menjaga keamanan data agar hanya field tertentu yang dapat diisi oleh pengguna.

## D. Controller dan Route Resource

```
php artisan make:controller ProductController --resource
```

Controller pada Laravel dibuat dengan tipe **resource** untuk mendukung operasi CRUD secara lengkap. Dengan menggunakan **route resource**, seluruh endpoint CRUD seperti menampilkan data, menambah data, mengubah data, dan menghapus data dapat dibuat secara otomatis tanpa harus menuliskannya satu per satu.

```
Route::resource('product', ProductController::class);
```

Controller berperan sebagai pengatur alur logika aplikasi, sedangkan route menghubungkan request dari user ke fungsi yang sesuai di controller.

## E. Validasi Form

```
$validated = $request->validate([  
    'name' => 'required|min:4',  
    'price' => 'required|integer|min:1000000',  
]);
```

Validasi form digunakan untuk memastikan data yang dimasukkan oleh pengguna sesuai dengan aturan yang telah ditentukan. Pada kasus ini, nama produk wajib diisi dengan minimal panjang karakter tertentu, sedangkan harga harus berupa angka dan memiliki nilai minimum tertentu. Jika data tidak memenuhi ketentuan, sistem akan secara otomatis menolak input dan memberikan pesan kesalahan kepada pengguna.

## MODUL 13

### A. Session

Session digunakan untuk menyimpan data sementara pengguna di server selama proses login berlangsung. Data ini dapat digunakan kembali pada halaman lain tanpa harus mengambil ulang dari database.

Setelah login berhasil, data seperti email dan nama user disimpan ke dalam session. Saat logout, seluruh data session dihapus untuk mengakhiri sesi pengguna.

```
session()->put('email', $u->email);  
session()->put('name', $u->name);  
session()->flush();
```

## B. Middleware

Middleware berfungsi sebagai lapisan keamanan untuk membatasi akses ke halaman tertentu. Middleware akan memeriksa apakah user sudah login sebelum mengizinkan akses ke route tertentu.

```
Route::resource('product', ProductController::class)->middleware('auth');
```

## C. Auth Laravel

Auth digunakan untuk proses autentikasi user seperti login dan logout. Laravel akan mencocokkan email dan password yang dimasukkan dengan data yang ada di database.

Jika data valid, maka user akan diarahkan ke halaman tertentu.

```
if (Auth::attempt(['email' => $req->em, 'password' => $req->pwd])) {  
    return redirect('/product');  
}
```

## D. Directive @auth

Directive **@auth** digunakan pada Blade template untuk menampilkan konten hanya kepada user yang sudah login. Jika user belum login, bagian tersebut tidak akan ditampilkan.

```
@auth  
    {{ Auth::user()->name }}  
    <a href="/logout">Logout</a>  
@endauth
```

## E. Model Relation

Relasi model digunakan untuk menghubungkan tabel dalam database. Pada kasus ini, satu **Product** dapat memiliki banyak **Variant**, sedangkan setiap **Variant** hanya dimiliki oleh satu **Product**.

```
// Product Model  
public function variants()  
{  
    return $this->hasMany(Variant::class);  
}  
  
// Variant Model  
public function product()  
{  
    return $this->belongsTo(Product::class);  
}
```

## UNGUIDED

### 1. Tugas 1 - Penjelasan Git Branch & Simulasi Git Branch

#### Penjelasan

Git adalah sistem pengelolaan versi yang digunakan untuk mencatat perubahan pada kode program. Salah satu fitur penting dalam Git adalah **branch**, yaitu cabang dari repository utama yang memungkinkan pengembang bekerja secara paralel tanpa mengganggu kode utama.

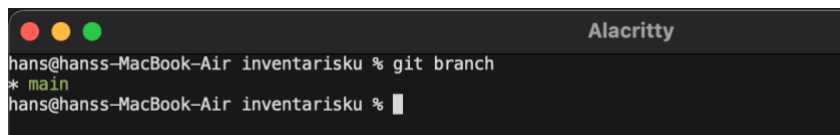
Branch digunakan untuk mengembangkan fitur baru, memperbaiki bug, atau melakukan eksperimen tanpa merusak branch utama (biasanya disebut **main** atau **master**). Setelah perubahan selesai dan stabil, branch tersebut dapat digabungkan kembali ke branch utama menggunakan proses **merge**.

Dengan menggunakan branch, pengembangan proyek menjadi lebih terstruktur karena setiap fitur dapat dikerjakan secara terpisah oleh masing-masing developer.

#### Simulasi

##### Simulasi Git Branch

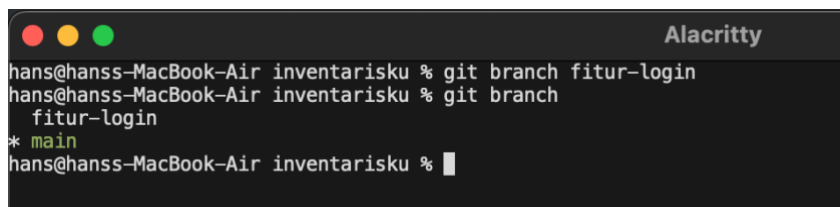
###### 1. Melihat branch yang ada



```
hans@hanss-MacBook-Air inventarisku % git branch
* main
hans@hanss-MacBook-Air inventarisku %
```

Perintah ini digunakan untuk menampilkan daftar branch yang tersedia pada repository lokal.

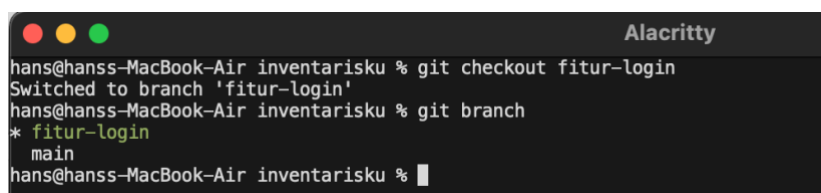
###### 2. Membuat branch baru



```
hans@hanss-MacBook-Air inventarisku % git branch fitur-login
hans@hanss-MacBook-Air inventarisku % git branch
  fitur-login
* main
hans@hanss-MacBook-Air inventarisku %
```

Perintah ini membuat branch baru bernama **fitur-login** tanpa langsung berpindah ke branch tersebut.

###### 3. Berpindah ke branch



```
hans@hanss-MacBook-Air inventarisku % git checkout fitur-login
Switched to branch 'fitur-login'
hans@hanss-MacBook-Air inventarisku % git branch
* fitur-login
  main
hans@hanss-MacBook-Air inventarisku %
```

Perintah ini digunakan untuk berpindah dari branch saat ini ke branch **fitur-login**.

#### 4. Melakukan perubahan dan commit

```
hans@hanss-MacBook-Air inventarisku % git status
On branch fitur-login
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        deleted:    laporan.md

no changes added to commit (use "git add" and/or "git commit -a")
hans@hanss-MacBook-Air inventarisku % git add .
hans@hanss-MacBook-Air inventarisku % git commit -m "update: readme & rm laporan.md"
[fitur-login 174bd60] update: readme & rm laporan.md
2 files changed, 1 insertion(+), 173 deletions(-)
delete mode 100644 laporan.md
hans@hanss-MacBook-Air inventarisku % git push
fatal: The current branch fitur-login has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin fitur-login

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
hans@hanss-MacBook-Air inventarisku % git push --set-upstream origin fitur-login
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 310 bytes | 310.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'fitur-login' on GitHub by visiting:
remote:   https://github.com/YamaTaro38/inventarisku/pull/new/fitur-login
remote:
To github.com:YamaTaro38:YamaTaro38/inventarisku.git
 * [new branch]      fitur-login -> fitur-login
branch 'fitur-login' set up to track 'origin/fitur-login'.
hans@hanss-MacBook-Air inventarisku %
```

Perintah ini menyimpan perubahan ke dalam branch yang sedang aktif.

#### 7. Menggabungkan branch (merge)

```
hans@hanss-MacBook-Air inventarisku % git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
hans@hanss-MacBook-Air inventarisku % git merge fitur-login
Updating add36d9..174bd60
Fast-forward
 README.md | 2 +-
 laporan.md | 172 -----
 2 files changed, 1 insertion(+), 173 deletions(-)
 delete mode 100644 laporan.md
hans@hanss-MacBook-Air inventarisku %
```

Perintah ini menggabungkan isi branch **fitur-login** ke branch **main**.

## 2. Tugas 2 – Laravel Databse 1 & 2

Aplikasi **InventarisKu** ini adalah project Laravel 11 untuk manage data produk. Ada fitur login/register, CRUD produk, dan database MySQL. Setiap user cuma bisa lihat & manage produknya sendiri.

Yang diimplementasikan:

- Auth (register, login, logout) pakai Laravel Breeze
- CRUD produk (tambah, lihat, edit, hapus)
- Database MySQL dengan relasi user ke produk
- Tampilan neurobrutalist (hitam-putih dengan aksen merah)

#### Database Konfigurasi

Setting database di file .env:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=inventarisku
DB_USERNAME=root
DB_PASSWORD=
```

## Struktur Tabel

Ada 2 tabel utama yang dibuat pake migration Laravel:

**Tabel Users** - nyimpen data user:

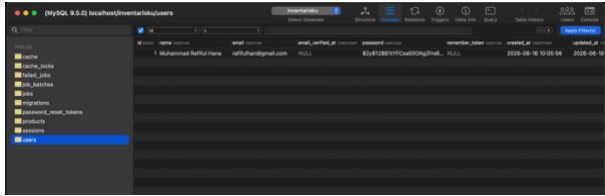
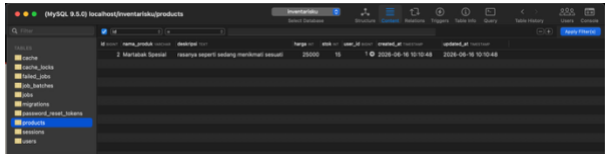
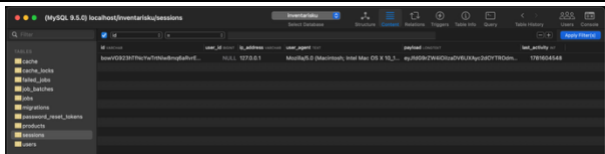
| Kolom      | Type      | Kegunaan                   |
|------------|-----------|----------------------------|
| id         | bigint    | Primary key                |
| name       | varchar   | Nama user                  |
| email      | varchar   | Email (unique, buat login) |
| password   | varchar   | Password (di-hash)         |
| created_at | timestamp | Waktu daftar               |
| updated_at | timestamp | Waktu update               |

**Tabel Products** - nyimpen data produk:

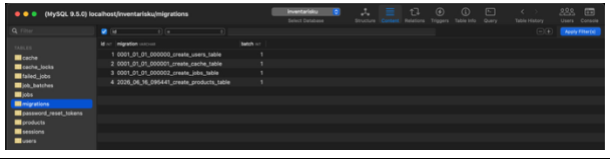
| Kolom       | Type      | Kegunaan                 |
|-------------|-----------|--------------------------|
| id          | bigint    | Primary key              |
| nama_produk | varchar   | Nama produk              |
| deskripsi   | text      | Deskripsi (boleh kosong) |
| harga       | integer   | Harga produk             |
| stok        | integer   | Jumlah stok (default 0)  |
| user_id     | bigint    | Foreign key ke users.id  |
| created_at  | timestamp | Waktu input              |
| updated_at  | timestamp | Waktu update             |

Relasi: products.user\_id nyambung ke users.id. Kalau user dihapus, produknya ikut kehapus (cascade).

## Tampilan di Database (SequelAce)

| Tabel    | Screenshot                                                                           |
|----------|--------------------------------------------------------------------------------------|
| Users    |  |
| Products |  |
| Sessions |  |



| Tabel      | Screenshot                                                                         |
|------------|------------------------------------------------------------------------------------|
| Migrations |  |

## Autentikasi (Login/Register)

Pake **Laravel Breeze** - tinggal install langsung jadi.

### Halaman Register & Login

| Halaman  | Screenshot                                                                        | Penjelasan                                                  |
|----------|-----------------------------------------------------------------------------------|-------------------------------------------------------------|
| Register |  | Form daftar: isi nama, email, password, konfirmasi password |
| Login    |  | Form masuk: isi email & password                            |

### Route yang diamankan

Semua route produk pake middleware auth:

```
Route::middleware('auth')->group(function () {
    Route::resource('products', ProductController::class);
});
```

Artinya: kalau belum login, gabisa akses halaman produk. Bakal diarahin ke halaman login.

## CRUD Produk

### Model Produk

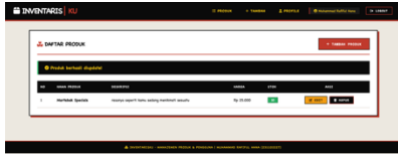
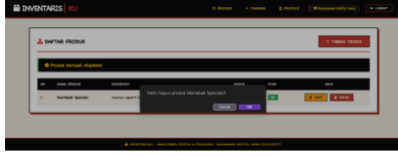
```
class Product extends Model
{
    protected $fillable = ['nama_produk', 'deskripsi', 'harga', 'stok', 'user_id'];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

### Biar data terisolasi per user

- Waktu lihat daftar: cuma tampilkan produk where('user\_id', Auth::id())
- Waktu edit/update/hapus: dicek dulu, kalo bukan punya bakal forbidden (403)

### Tampilan Fitur CRUD

| Fitur         | Screenshot                                                                           | Penjelasan                                                     |
|---------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Dashboard     |    | Daftar produk dalam tabel. Ada tombol Tambah Produk            |
| Tambah Produk |    | Form isi nama, deskripsi, harga, stok                          |
| Hasil Tambah  |    | Muncul alert hijau "berhasil", data baru langsung ada di tabel |
| Edit Produk   |   | Form yang sudah terisi data lama, tinggal diubah               |
| Hasil Edit    |  | Data berubah di tabel, ada alert sukses                        |
| Hapus         |  | Tinggal klik tombol hapus, pake method DELETE                  |
| Hasil Hapus   |  | Produk ilang dari tabel, ada alert sukses                      |